

Brief Introduction to Python Programming



Google Colab used for Examples



Introduction to Python 3

- Python is a high-level programming language
- Open source and community driven
- A standard distribution includes many modules
- Dynamic typed
- Source can be compiled or interpreted (run just-in-time)
- Similar to perl, tcl, ruby

- We are using **Python 3**, not 2

Python: an OO language

- Practically everything can be treated as an object
- For example, “a string” is a string object
- Strings, as objects, have methods that return the result of a function on the string

String Methods

- To assign a string to a variable, “str”

```
str = “a string”
```

- `str.title()` # A String
- `str.upper()` # A STRING
- `str.isdigit()` # false
- `Str.isalpha()` # true
- `str.islower()` # true

String Methods

- The string held in your variable remains the same, a string obj is ***immutable***
- The method returns an altered string
- Changing the variable requires **re**-assignment
 - `str = str.upper()`
 - str now equals "A STRING"

Other Python Objects

- **Lists** (mutable sets)

```
l = [] # create list
l = ['one', 2, 'three', 'apple']
Print(l[0])    # one will be printed
```

- **Tuples** (immutable sets)

```
t = ('one', 2, 'three', 'apple')
```

- **Dictionaries** (associative arrays or hashes, mutable)

```
var = {} # create dictionary
var = {'abc': 321.20, 'xyz': -128.72}
print(var['abc']) # 321.20 will be printed
```

2D Lists (Examples using Google Colab)



```
l2 = [[1,2,3], [4,5,6]]  
print (l2[0])  
print (l2[1][1])
```

```
[1, 2, 3]
```

```
?
```

Jagged Array

```
# Creating a 2D array with different row sizes
```

```
my_array = [  
    [1, 2, 3],  
    [4, 5],  
    [6, 7, 8, 9]  
]
```

```
# Accessing elements
```

```
print(my_array[0])
```

```
print(my_array[1][1])
```

```
# Finding the size of each row
```

```
for row in my_array:
```

```
    print(len(row))
```

Output

[1, 2, 3]

5

3

2

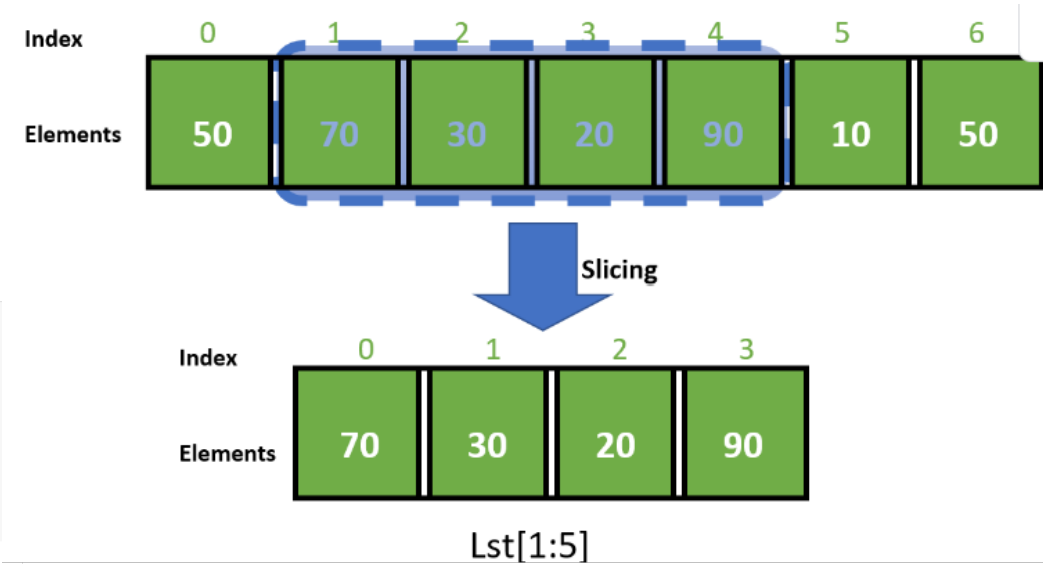
4

List Slicing



```
Lst = [50, 70, 30, 20, 90, 10, 50]  
print(Lst[1:5])
```

```
[70, 30, 20, 90]
```



More slicing later in Numpy

<https://www.geeksforgeeks.org/python-list-slicing>

Python Numbers

- There are three numeric types in Python:
 - int
 - float
 - complex
- They are “immutable (unchangeable)” objects – see the next slide.

```
x = 1      # int
y = 2.8    # float
z = 1+1j   # complex
print(type(x))
print(type(y))
print(type(z))

a = float(x) #convert from int to float
print(a)

b = int(y)   #convert from float to int
print(b)

c = complex(x) #convert from int to complex:
print (c)
```

```
<class 'int'>
<class 'float'>
<class 'complex'>
1.0
2
(1+0j)
```

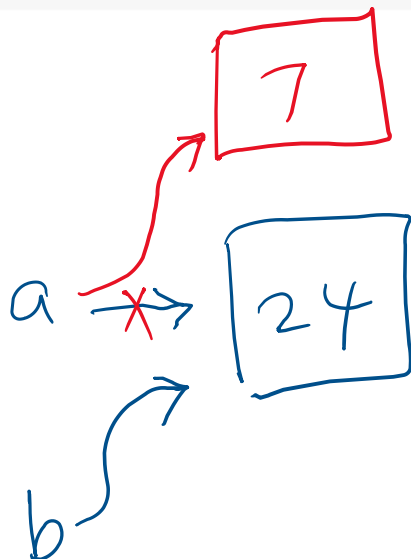
Numbers are immutable!!

```

▶ a = 24
  b = a # now both a and b are equal
  a = 7
  print(b) # 7? No!
  print(a)

```

↗ 24
 7



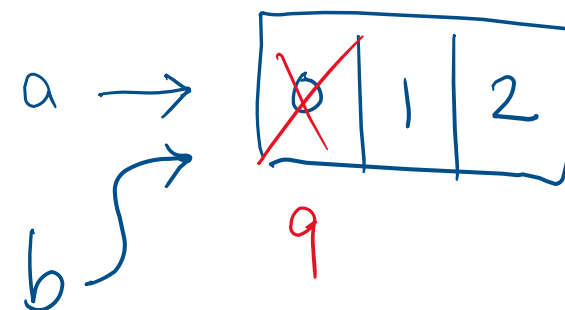
cf. mutable list

```

▶ a = [0, 1, 2]
  b = a
  a[0] = 9
  print(b)

```

↗ [9, 1, 2]

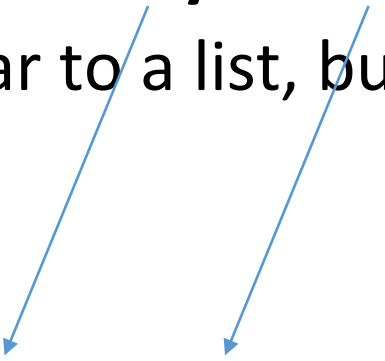


Tuples

- Like a list, tuples are iterable arrays of objects
- Tuples are *immutable* – once created, unchangeable
- To add or remove items, you must re-declare
- Example uses of tuples
 - County Names
 - Land Use Codes
 - ...

Dictionaries

- Dictionaries are sets of **key & value** pairs
- A dictionary is similar to a list, but you access values by looking up a **key** instead of an index
- Mutable objects



```
residents = {'A' : 104, 'B' : 105, 'C' : 106}  
print(residents['B']) # Prints B's room number
```



Decision Making

- **if and else**

```
if variable == condition:  
    #do something based on v == c  
else:  
    #do something based on v != c
```

- **elif allows for additional branching**

```
if condition:  
elif another condition:  
...  
else: # none of the above
```

Shorthand If Else



```
# Shorthand (Shortcut) If Else  
a = 2  
b = 330  
print(a) if a > b else print(b)
```



```
330
```

Cf. C++

```
(a > b)? print(a) : print(b);
```

Indentation and Blocks

- Python uses whitespaces (indents) to denote blocks of code (**Do not mix tabs and spaces**)
- Lines within the code block are indented at the same level
- To end a code block, remove the indentation

```
i = int(input("Enter a number: "))
if i < 0:
    print("Example code to show blocks.")
    print("The number is negative.")
else:
    print("The number is nonnegative.")
    if i < 10:
        print ("The number has one digit.")
    else:
        print ("The number has multiple digits.")
print("end")
```




```
# Python program to demonstrate ">", and logical "and" operator
```

```
a = 10
```

```
b = 10
```

```
c = -10
```

```
if a > 0 and b > 0:
```

```
    print("The numbers are greater than 0")
```

```
if a > 0 and b > 0 and c > 0:
```

```
    print("The numbers are greater than 0")
```

```
else:
```

```
    print("At least one number is not greater than 0")
```



The numbers are greater than 0

At least one number is not greater than 0



```
# to demo "not", "or", and "=="  
a = 10  
if not a:  
    print("Boolean value of a is True")  
  
if not (a%3 == 0 or a%5 == 0):  
    print("10 is not divisible by either 3 or 5")  
else:  
    print("10 is divisible by either 3 or 5")
```

10 is divisible by either 3 or 5

Looping with “for”

for allows you to loop over a block of code a fixed number of times

Iterable object like a list or a range()

```
list = ['smart', 'hardwork', 'fun']  
for x in list:    # do not forget ":"  
    print(x, len(x));
```

```
smart 5  
hardwork 8  
fun 3
```

Semi colon at the end is OK in Python like C style.

For loop example



```
A = [1,2,3,4,5]
```

```
s = 0
```

```
for n in A:
```

```
    s = s + n
```

```
print (s)
```

```
print(sum(A))
```

15

15

For loop example with a list of lists



```
A = [[1,2], [3,4], [5,6]]  
s = 0  
for x,y in A:  
    s += x+y  
print(s)
```



Iterate over a Dictionary with For Loop



```
# Iterate Over a Dictionary with For Loop
residents = {'Tom':101, 'Bob': 211, 'John':217}
for key, value in residents.items():
    print(key, value, end=" ")
print()
for key in residents:
    print(key, residents[key], end=" ")
```

```
☞ Tom 101 Bob 211 John 217
   Tom 101 Bob 211 John 217
```

For : break (breaks control out of the loop)



```
# break
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    print(x)
    if x == "banana":
        break
```



```
apple
banana
```

For : continue (Returns the control to the beginning of the loop) Do not print banana



```
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    if x == "banana":  
        continue  
    print(x)
```

```
apple  
cherry
```


For : pass

- Case for doing nothing
- Usually a **placeholder** for future code to avoid getting an error, since empty code is not allowed in loops, function definitions, class definitions, or in if statements.

```
for x in [0, 1, 2]:  
    pass  
  
def myfunction():  
    pass
```

```
b = 3  
if b > 0:  
    pass  
else:  
    print(b)
```

For loop and *Iterator*

- Iterator is an object used to iterate over iterable objects
- Iterator is commonly used to loop through the objects of a collection, read them, and ***remove*** them.

Tuple must be used. List cannot be used here

```
[64] fruits = ["apple", "banana", "cherry", "berry", "mellon"]
      iterator1 = (f.upper() for f in fruits)
      print(next(iterator1)) # the first item processed (printed) and removed
      for f in iterator1: # the rest items are processed
          print(f, end=' ')
```

```
➞ APPLE
   BANANA CHERRY BERRY MELLON
```

Looping with “while” and print formatting

```
▶ n = 10

sum = 0
counter = 1
while counter <= n:
    sum = sum + counter
    counter += 1

print("Sum of 1 + ... + %d = %d" % (n,sum)) # old style formatting
print("Sum of 1 + ... + {0} = {1}".format(n, sum)) # I do not like it
print(f"Sum of 1 + ... + {n} = {sum}") # Python 3.6+ 
```

```
☞ Sum of 1 + ... + 10 = 55
   Sum of 1 + ... + 10 = 55
   Sum of 1 + ... + 10 = 55
```

No “do while” in Python

No “Do-While” in Python



Do-While(condition)

Emulating the “Do-While” in Python

```
while True:
    # Execute your code here. “do”
    if not condition:
        break
```

example

```
while True:
    user_input = input("Enter a positive number: ")
    if user_input.isdigit() and int(user_input) > 0:
        break
    print("Invalid input. Please try again.")
```

Fixed digits after decimal with f-strings

```
a = 3.141592
print(f"{a:.3f}")
print(f"{a:5.2f}") # width.precision.
                    # width: the # of characters used in total to display
```

```
3.142
3.14
```

Range function

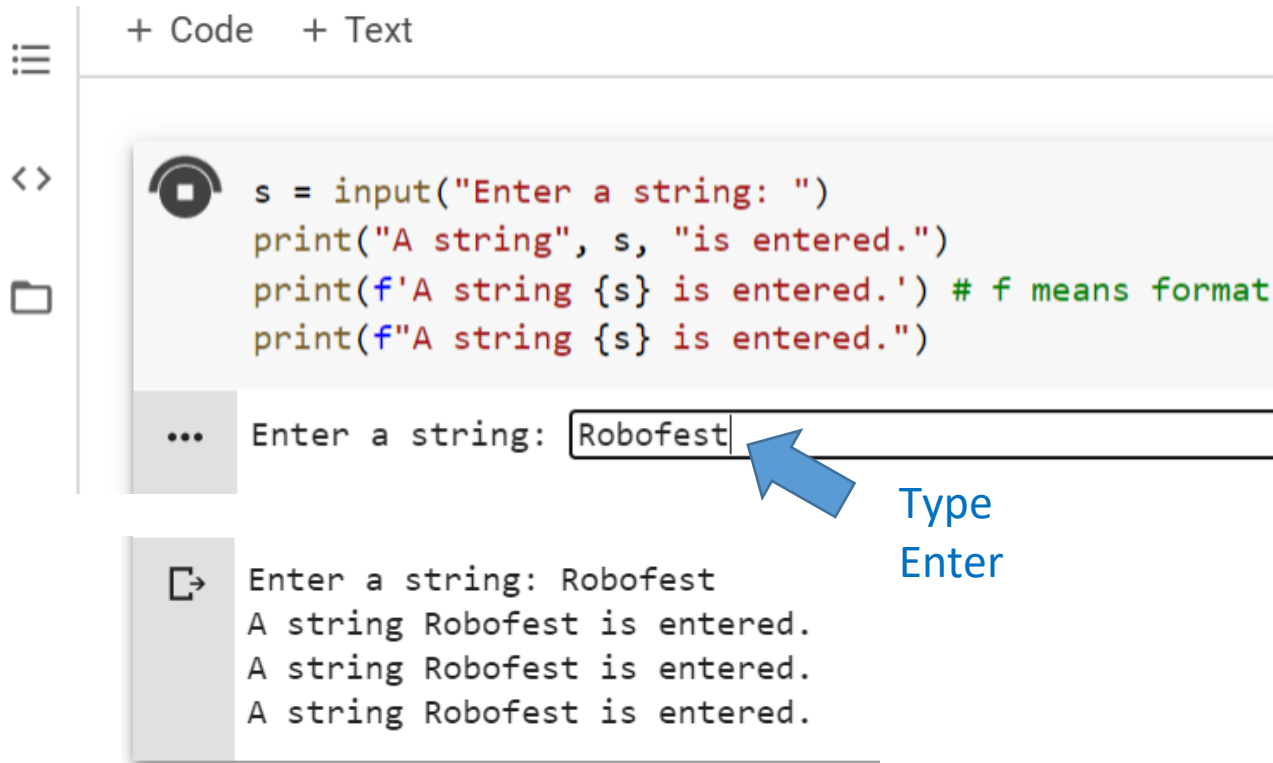


```
a = ['Mary', 'had', 'a', 'little', 'lamb']  
for i in range(len(a)): # range will generate 0, 1, 2, 3, 4  
    print(i, a[i])
```



```
0 Mary  
1 had  
2 a  
3 little  
4 lamb
```

Python user input from Keyboard: `input(prompt)`



The screenshot shows a Jupyter Notebook interface. At the top, there are tabs for '+ Code' and '+ Text'. Below the tabs, there is a code cell containing the following Python code:

```
s = input("Enter a string: ")
print("A string", s, "is entered.")
print(f'A string {s} is entered.') # f means format
print(f"A string {s} is entered.")
```

Below the code cell, there is an input field with the text "Enter a string: Robofest". A blue arrow points to the input field with the text "Type Enter".

Below the input field, there is an output cell showing the following output:

```
Enter a string: Robofest
A string Robofest is entered.
A string Robofest is entered.
A string Robofest is entered.
```

} Using Python f-strings,
available 3.6+

Converting string to integer in Python

```
 str = input("Enter feet: ")  
i = int(str)  
print(f"{str}ft = {i*12} inches")
```

... Enter feet:

```
 Enter feet: 3  
3ft = 36 inches
```

Example: Nested for loops with range

```
▶ n = int(input("Enter a number: "))  
  for i in range(n):  
    for j in range(i+1):  
      print('X', end='') # end='' to print without newline  
    print()
```

```
☞ Enter a number: 10  
X  
XX  
XXX  
XXXX  
XXXXX  
XXXXXX  
XXXXXXX  
XXXXXXXX  
XXXXXXXXX  
XXXXXXXXXX  
XXXXXXXXXX
```

Exception Handling: “try except”

```
while True:
    try:
        n = int(input("Enter a positive number: "))
        if (n < 1):
            raise Exception(f'{n} is not a positive int.')
        for i in range(n):
            for j in range (i+1):
                print('X', end='')
            print()
        break      # break the while loop
    except Exception as e:
        print(f"{e}") # continue
```

```
Enter a positive number: y
invalid literal for int() with base 10: 'y'
Enter a positive number: -3
-3 is not a positive int.
Enter a positive number: 3
X
XX
XXX
```

Importing modules in Python

```
import math # math module library  
print(math.pi)
```

3.141592653589793

```
import math as m  
print(m.pi)  
print(m.sqrt(5))
```

3.141592653589793
2.23606797749979

Importing modules in Python

```
from math import pi
# Note that in the above example, we used math.pi.
# Here we can use pi directly.
print(pi)
#print(factorial(5)) # Not imported
```

3.141592653589793

```
from math import *
print(pi)
print(factorial(5))
print(sqrt(3))
```

3.141592653589793

120

1.7320508075688772

Defining & Calling Functions of Your Own

```
# Define a function `plus()`  
def plus(x,y):  
    answer = x + y  
    return answer  
  
def multiply(x,y):  
    answer = x * y  
    return answer  
  
print (plus(2,3))  
print (multiply(3,4))
```

parameters

Positional arguments

```
5  
12
```

No return type is needed to define in Python!

Returning Multiple Values in Python

- Object
- Tuple
- List
- Dictionary



```
def calc(x, y):  
    plus = x + y  
    mul = x * y  
    return plus, mul  
  
print (calc(2,3))  
p,m = calc(4,5)  
print (p, m)
```

```
(5, 6)  
9 20
```

Return Type Hints

-> int indicates that the function is expected to return an integer.



```
def add_numbers(x: int, y: int) -> int:  
    return x + y
```

```
add_numbers(10, 20)
```



30

Passing Arguments (1)

- Python uses a mechanism, which is known as "Call-by-Object", sometimes also called "Call by Object Reference"
- Mutable arguments, i.e., lists are also passed by object reference, and they can be changed in place within the function.
- If you pass *immutable* arguments like integers, floats, strings or tuples to a function, the passing acts like call-by-value. The object reference is passed to the function parameters. But they can't be changed within the function, because they can't be changed at all, they are immutable!



```
def append_n(list, n):  
    list[0] = 77  
    list.append(n)
```

```
mylist = [0,1,2]  
num = 9  
append_n(mylist, num)  
print(mylist)  
print(num)
```

mutable

```
[77, 1, 2, 9]  
9
```

Passing Arguments (2)

- Python uses a mechanism, which is known as "Call-by-Object", sometimes also called "Call by Object Reference"
- Mutable arguments, i.e., lists are also passed by object reference, and they can be changed in place within the function.

- If you pass immutable arguments like integers, floats, strings or tuples to a function, the passing acts like call-by-value. The object reference is passed to the function parameters. But they can't be changed within the function, because they can't be changed at all, they are immutable!



n is a new int object created in this function

```
def append_n2(list, n):  
    n += 1  
    list.append(n)  
  
mylist = [0,1,2]  
num = 9  
append_n2(mylist, num)  
print(mylist)  
print(num)
```

immutable

[0, 1, 2, 10]
9

Keyword Arguments

```
▶ def my_function(child3, child2, child1):  
    print("The youngest child is " + child3)  
  
my_function("Mee", child1 = "Young", child2 = "Key")
```

⇒ The youngest child is Mee

Positional arg

keyword arg

Arbitrary Positional Args (variable length args)

args are wrapped up in a tuple; no keyword args!

```
 def my_sum(*apa):  
    result=0  
    for i in apa:  
        result=result+i  
    return result  
  
print (my_sum(1,2,3,4,5))  
print (my_sum(10,20))
```

```
 15  
30
```

Arbitrary Keyword Arguments – to hold keyword variable-length arguments



```
def fn(**a):  
    for i in a.items():  
        print (i)
```

```
fn(numbers=153, colors="green", fruits="cherry")
```



```
('numbers', 153)  
( 'colors', 'green')  
( 'fruits', 'cherry')
```

List Comprehension: provide a concise way to create lists based on existing iterables

```
[4] numbers = [1,2,3,4,5,6,7]
    squared = [x*x for x in numbers]
    print(squared)
```

```
[1, 4, 9, 16, 25, 36, 49]
```



```
[5] # Using for loop
    squared = []
    for i in numbers:
        squared.append(i*i)
    print(squared)
```

```
[1, 4, 9, 16, 25, 36, 49]
```

List comprehension with `if` condition:

provide a concise way to create *lists* based on existing iterables

```
numbers = [1,2,3,4,5,6,7]
evens = [x for x in numbers if x % 2 == 0]
print(evens)
```

[2, 4, 6]

```
evens = []
for i in numbers:
    if i % 2 == 0:
        evens.append(i)
print(evens)
```

[2, 4, 6]

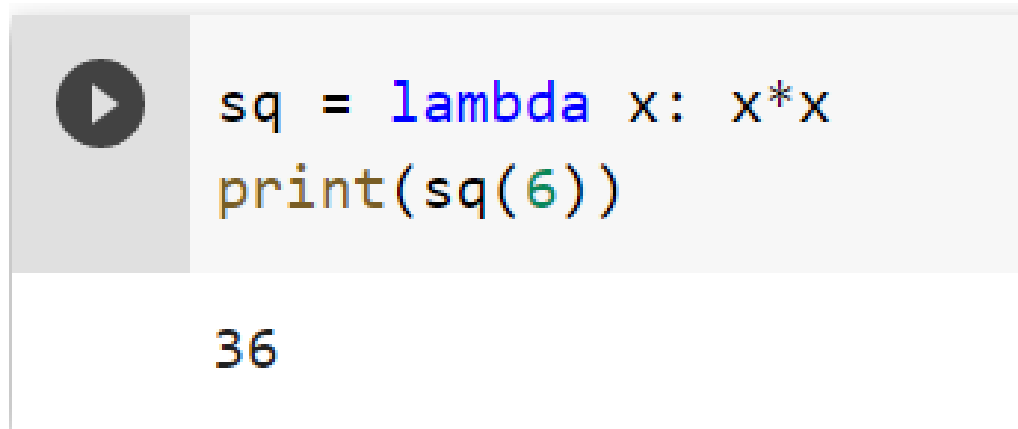
[expr for element in iterable if condition]

Above is equivalent to –

```
for element in iterable:
    if condition:
        expr
```

Python Anonymous/Lambda Function

- While normal functions are defined using the `def` keyword in Python, anonymous functions are defined using the **lambda** keyword without a name.
- Syntax of Lambda function `lambda argument(s): expression`

A code execution snippet showing a lambda function definition and its use. The code is: `sq = lambda x: x*x` followed by `print(sq(6))`. The output of the code is `36`.

```
sq = lambda x: x*x
print(sq(6))

36
```


Python Anonymous/Lambda Function

- lambda functions are used when we require a nameless function for a short period of time
- Example use with `filter(func, iterable)`
 - The `filter()` function in Python takes in a function and an iterable / list as

```
# to filter out only even numbers from a list, using a lambda f
my_list = [1,2,3,4,5,6,7,8,9]
even_list = list(filter(lambda x: (x%2 == 0), my_list))
print(even_list)
```

```
[2, 4, 6, 8]
```

<https://www.geeksforgeeks.org/lambda-filter-python-examples/>

Python Numpy Library

- Array oriented computing
- *Linear Algebra*
- Efficiently implemented multi-dimensional arrays
- Designed for scientific computation
- Used for plotting, **OpenCV**, Tensorflow, and Keras
- Pronounced /'nʌmpaɪ/ (NUM-py) or sometimes /'nʌmpi/ (NUM-pee))

Numpy Examples

```
import numpy as np          # people use "np"
celcius = [20, 21, 22, 23, 34, 36]
fahrenheit = [ x*9/5 + 32 for x in celcius ]
print(fahrenheit)
```

```
[68.0, 69.8, 71.6, 73.4, 93.2, 96.8]
```

```
# better way using numpy
C = np.array(celcius) # C is not a list. We call it as an array
F = C*9/5 + 32
print(F)
```



Linear algebra style

```
[68.  69.8 71.6 73.4 93.2 96.8]
```

What's wrong?



```
# What's wrong?  
import numpy as np  
Deg = [90, 180, 360]  
Rad = Deg*3.14/180  
print(Rad)
```

Corrected code



```
import numpy as np  
Deg = np.array([90, 180, 360])  
Rad = Deg*3.14/180  
print(Rad)
```

```
[1.57 3.14 6.28]
```

Numpy.dot product



```
vec1 = np.array([0.5, 1])  
vec2 = np.array([0, 3])  
print(vec1.dot(vec2))  
print(np.dot(vec1, vec2))
```

3.0

3.0

$$a \cdot b = \sum_{i=1}^n a_i b_i$$

a = 1st vector

b = 2nd vector

n = dimension of the vector space

a_i = component of vector a

b_i = component of vector b

Numpy.random.rand()



```
R = np.random.rand(3,4) # 3 (row) x 4 (column) array  
print(R)
```



```
[[0.73923298 0.15442509 0.47035514 0.48749334]  
 [0.34616891 0.88770246 0.51895347 0.94269328]  
 [0.68316983 0.88505089 0.7917706  0.48852444]]
```

Numpy array “append”

```
narr = np.array([])
for i in range(5):
    narr = np.append(narr, i)
print(narr)
```

```
[0.  1.  2.  3.  4.]
```


`numpy.where(cond)` - returns the *indices* of elements in an input array where the given condition is satisfied

```
import numpy as np
a = np.array([1,2,3,4,5,6])
print (np.where(a<3))
```

```
(array([0, 1]),)
```

```
print( a[np.where(a<3)] )
```

?

```
print(a[[0,1]])
```

```
[1 2]
```

Slicing (Same idea for string, list, Numpy, and tuple)

- <https://www.geeksforgeeks.org/python-list-slicing/>
- <https://stackoverflow.com/questions/509211/understanding-slicing>
- <https://stackoverflow.com/questions/31633635/what-is-the-meaning-of-inta-1-in-python>

Numpy array indexing & slicing



```
import numpy as np
```

```
a = np.array([1, 2, 3, 4, 5])
```

```
print(a[0])
```

```
print(a[:3]) # array slicing, from index 0 to index 2
```

```
print(a[3:]) # from index 3 till the end
```

```
print(a[2:4]) # from index 2 to 3
```

1

[1 2 3]

[4 5]

?

Numpy array indexing & slicing, more examples



```
C = np.array([0,1,2,3,4,5])
print("Original array", C)
print(C[0:2])
print(C[0:])
print(C[:])
print(C[:3])
print(C[1:2])
print(C[:-1]) # excluding last item
print(C[1:-1]) # excluding first & last item
print(C[1:-2]) # excluding last 2 items
print(C[-1]) # last item
```

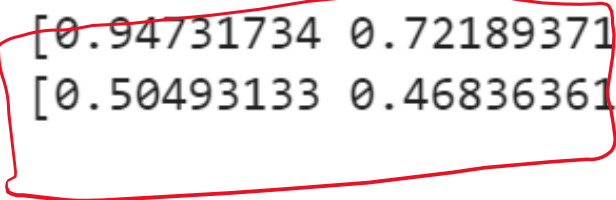


```
Original array [0 1 2 3 4 5]
[0 1]
[0 1 2 3 4 5]
[0 1 2 3 4 5]
[0 1 2]
[1]
[0 1 2 3 4]
[1 2 3 4]
[1 2 3]
5
```

Slicing 2D arrays

```
R = np.random.rand(3,4) # 3 (row) x 4 (column) array  
print(R)
```

```
[[0.39642695 0.15111664 0.5745798  0.90828164]  
 [0.94731734 0.72189371 0.8705027  0.20240275]  
 [0.50493133 0.46836361 0.23997483 0.01992207]]
```



```
print(R[1:3,0:2])
```

```
[[0.94731734 0.72189371]  
 [0.50493133 0.46836361]]
```

Local & Global variables

```
▶ name = 'cj'
def example():
    print(name)

example()
```

↗ cj

```
▶ # what's wrong with this code?
name = 'Rob'
def example():
    print(name)
    name = 'Bob'

example()
```



UnboundLocalError:

local variable 'name' referenced before assignment

```
▶ # fixed
name = 'Rob'
def example():
    global name
    print(name)
    name = 'Bob'

example()
```

↗ Rob

Of course you can fix this way too



```
name = 'Rob'  
def example():  
    name = 'Bob'  
    print(name)  
  
example()
```

Bob

Local & Global Variables

```
a = 1

def f():
    print('Inside f(): ', a) # Uses global because there is no local 'a'
    print()

def g():
    a = 2
    print(f'Inside g(): {a}') # Variable 'a' is redefined as a local
    print()

def h():
    global a
    a = 3
    print(f'Inside h(): {a}') # Uses global keyword to modify global 'a'
    print()

# Global scope
print('global : ', a)
f()
print('global : ', a)
g()
print('global : ', a)
h()

print('global : ', a)
```



```
global : 1
Inside f(): 1

global : 1
Inside g(): 2

global : 1
Inside h(): 3

global : 3
```


Python Inner (or Nested) Functions

- A function which is defined inside another function is known as inner function or nested function.
- Nested functions are able to access variables of the enclosing scope.
- Inner functions are used so that they can be protected from everything happening outside the function. Encapsulation without classes!
- <https://realpython.com/inner-functions-what-are-they-good-for/>
- <https://www.geeksforgeeks.org/python-inner-functions/>

Python Inner (or nested) Functions

```
▶ def outerFunction(text):  
    text = text  
  
    def innerFunction():  
        print(text)  
  
    innerFunction()  
  
#if __name__ == '__main__':  
outerFunction('Hey !')
```

Hey !

Python Inner (or nested) Functions

```
 def f1():  
    s = 'I love Python'  
  
    def f2():  
        s = 'Me too'  
        print(s)  
  
    print(s)  
    f2()  
  
# Main  
f1()
```

```
 I love Python  
Me too
```

Classes, Objects, and Methods

```
class Person:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def myGetMethod(self):
        print(f"Hello my name is {self.name}")

    def mySetMethod(self, n):
        self.salary += n
        print(f"My adjusted salary is ${self.salary}")

p1 = Person("John", 8000)
p1.myGetMethod()
p1.mySetMethod(300)
```

```
Hello my name is John
My adjusted salary is $8300
```

Print in **bold**



```
print("The bold text is", '\033[1m' + 'Python' + '\033[0m')
```

The bold text is **Python**

A Python package that generates LaTeX math description from Python functions

- https://github.com/google/latexify_py

$$\text{solve}(a, b, c) = \frac{-b + \sqrt{b^2 - 4ac}}{2a}$$

Sample Google Colab codes used in the slides are available on Canvas

- Intro_Python1.ipynb
 - Intro_Python2.ipynb
 - Intro_Python3.ipynb
 - Numpy_intro.ipynb
-
- File suffix “ipynb” stands for Interactive Python NoteBook

RQ: axis

```
threeBy2 = np.array([[0,1], [3,2], [0,3]])  
  
print(threeBy2.mean())  
print(threeBy2.mean(axis=0)) # row is collapsed  
print(threeBy2.mean(axis=1)) # col is collapsed
```

```
1.5  
[1. 2.]  
[0.5 2.5 1.5]
```

0	1
3	2
0	3

axis=0

0	1
3	2
0	3

axis=1

RQ: axis

```
threeBy4 = np.array([[0,0,1,0], [0,0,0,1], [0,1,1,2]])

print(np.argmax(threeBy4, axis=0))
print(np.argmax(threeBy4, axis=1))
```

[0 2 0 2]
[2 3 3]

0	0	1	0
0	0	0	1
0	1	1	2

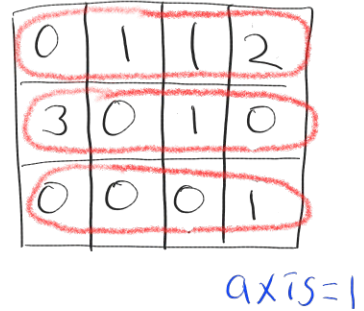
axis=0

0	0	1	0
0	0	0	1
0	1	1	2

axis=1

RQ: np.where(), np.argmax(), and axis

```
aa = np.array([[0,1,1,2], [3,0,1,0], [0,0,0,1]])
bb = np.array([[0,1,1,2], [3,0,1,0], [0,0,3,0]])
print("A = ", np.argmax(aa, axis=1))
print("B = ", np.argmax(bb, axis=1))
i = np.where(np.argmax(aa, axis=1) != np.argmax(bb, axis=1))[0]
print("C = ", i)
```



0	1	1	2
3	0	1	0
0	0	0	1

axis=1

```
A = [3 0 3]
B = [3 0 2]
C = [2]
```